MERU · PERSONAL AI ASSISTANT · WRITTEN IN GO · PRE-ALPHA

AN AI ASSISTANT ON YOUR OWN MACHINE

Meru runs open-weight models on a machine you control. You need no API key and pay for no tokens, so a question costs electricity, and the model on your disk today still runs a year from now. No prompt leaves the machine unless you allow it.

github.com/aarora79/meru

Architecture in three levels: **aarora79.github.io/meru/**

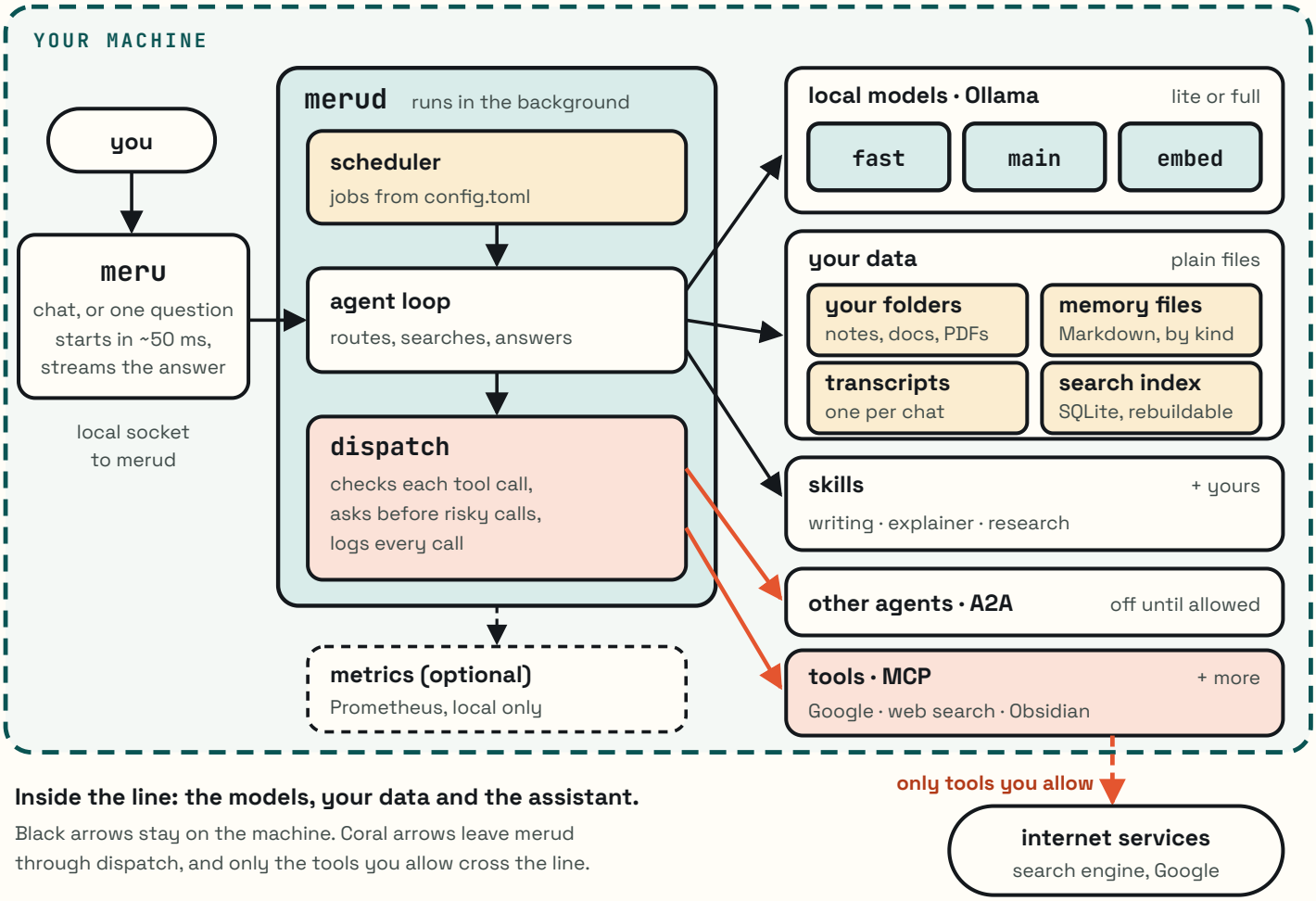


मेरु

In some Eastern traditions, **Meru** is the cosmic mountain that the sun, moon and stars turn around. This assistant takes the name because it works the same way: it stays in one place, on your machine, and your notes, tools and daily routine turn around it.

01 EVERYTHING RUNS INSIDE THE DASHED LINE

the parts of Meru



Inside the line: the models, your data and the assistant.

Black arrows stay on the machine. Coral arrows leave merud through dispatch, and only the tools you allow cross the line.

02 PICK THE PROFILE YOUR MACHINE CAN HOLD

lite is the default

TIER	JOB	LITE (DEFAULT)	FULL
`fast`	routing, trivial answers	MiniCPM5-2B, 1.6 GB	same
`main`	reasoning, choosing tools	same model as `fast`	`qwen3.6:35b-a3b-mxfp8`, ~47 GB loaded
`embed`	vectors for search	`embeddinggemma-2:440m`, text and images	same
`needs`	hardware	16 GB of RAM; a CPU is enough	a 64 GB Mac, or a GPU with ~47 GB

Platforms: macOS on Apple silicon (tested), Linux, cloud virtual machines included (tested on Ubuntu 24.04 with an NVIDIA card), and Windows 10+ (untested at first).

03 THE RULES THE CODE KEEPS

from the privacy boundary

- **No prompt to the internet.** A model you name on your own network gets a question only after your yes.
- **Deny by default.** A new MCP server or A2A agent gives the model nothing until you allow tools by name.
- **Every external action logged.** Review them all with `meru log`.
- **One anonymous count a day.** It holds no ID or text, and you can turn it off. No crash reports, no update checks.
- **Loopback unless you say so.** A remote server or agent needs `remote = true` in its entry.
- ! **No sandbox.** MCP servers run with your permissions, so add only ones you trust.

Pre-alpha, v0.7.0. The design came first, and the code follows it. On a Mac or on Linux, Meru answers from the folders you index and cites the files and pages it used.

04 ONE TURN, FIVE STEPS, IN A LOOP WE WROTE

no agent framework · traced locally

1 · Route
`fast` picks a route from one token.

2 · Context
Skills, memories, hits, history, each on a budget.

3 · Main
`main` streams the answer to you, or asks for a tool.

4 · Dispatch
Checks, asks if needed, runs and logs each call.

5 · Repeat
Until the answer needs no tool, or 8 rounds.

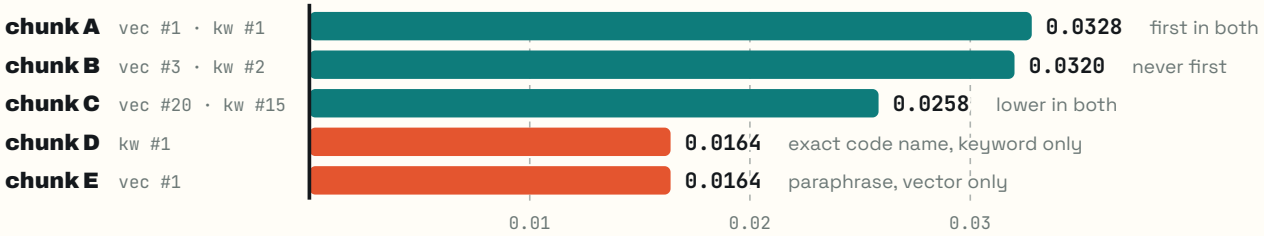
`merud` traces each step with OpenTelemetry and hands the data only to a program on this machine, such as a Prometheus you run, with no prompt text unless you opt in. The Usage page charts it with nothing to install.

05 TWO SEARCHES, MERGED BY RANK

BM25 + vec1 distance + reciprocal-rank fusion

Search covers your files, memories and past chats. Keyword search (BM25) finds exact strings such as code names; meaning search finds paraphrase. Go merges the two lists.

RRF SCORE = Σ 1/(60 + RANK) · RANKS INVENTED, SCORES COMPUTED



06 EVERY TOOL CALL PASSES ONE FUNCTION

dispatch · deny by default

allow? If not: denied. → **confirm?** If so: ask you. → **call** MCP, A2A or built-in. → **tool_calls** Log every outcome. → **span** Time and result.

`approve once`	This call runs. The next one asks again.
`this session`	No more asking about that tool until the chat ends.
`deny`	The call doesn't run, and the model hears that you said no.

`configure` and `schedule_job` ask every time. A script has no one to ask, so `dispatch` denies; a job runs what its `allow` list names.

```
[[mcp.servers]]
name    = "obsidian"
command = "uvx"          # stdio child
allow   = ["search", "read", "append"] # rest: off
confirm = ["append"]       # ask me first
```

07 MEMORY IS A FOLDER OF MARKDOWN FILES

~/.meru/memory/ · one fact per file

```
me/           who you are
preferences/  how you like things done
projects/     work, goals, deadlines
people/       people you mention
reference/    where things live
other/        anything else
```

- Each file carries its date and source chat. Delete the file and Meru forgets it.
- The model saves one with the built-in `remember` tool, through `dispatch`.
- Each chat ends with a short summary, so "what did we decide last week?" finds it.
- Files hold the truth. `meru.db`, one SQLite file with keywords and vectors, rebuilds from them.

08 SETUP TAKES ONE COMMAND

meru setup · skills · jobs

MERU SETUP · FIRST RUN

1. Checks for Ollama and prints the install command if needed.
2. Downloads the answer model that fits your Mac's memory or your GPU.
3. Writes `config.toml` and asks which folders to index.
4. Offers three starter MCP servers (Google, web search, Obsidian). It adds each for you, or shows you what to paste.
5. Answers a test question.

SKILLS AND JOBS

- `writing`, `explainer` and two research skills ship built in.
- Skills are Markdown in `~/.meru/skills/`. Your edits win.

```
[[jobs]]
name     = "morning-brief"
schedule = "0 7 * * 1-5"  # weekdays 7:00
prompt   = "Brief me on today..."
```

09 SHIPPING ORDER

all shipped · each useful by itself

v0.1 IT TALKS — First token under 1s.

v0.2 IT KNOWS YOUR FILES — Cites your notes.

v0.3 IT CAN DO THINGS — Calls MCP servers.

v0.4 IT KNOWS YOU — Recalls last week.

v0.5 IT ACTS UNPROMPTED — A brief, unasked.

v0.6 IT RUNS ON LINUX — One-command install.

Pre-alpha, v0.7.0. On a 64 GB Mac the `full` set passed 89% of 50 test tasks, with a first word in 4.1s. Panel 05's ranks are made up.

READ THE DESIGN, FOLLOW THE BUILD

Code: github.com/aarora79/meru — README.md, ARCHITECTURE.md, ROADMAP.md
Architecture in three levels: **aarora79.github.io/meru/**
Amit Arora · Apache-2.0 · Ollama on loopback · Go binaries you can copy to another machine and run.